

ECOM EXPERTS

Ecom ucpgoogle

Manual de usuario, tecnico y de desarrollo

ecom_ucpgoogle · 08/09/2026

Contenido

¿Qué hace este módulo?

Instalación rápida

Pestañas del panel

1. Onboarding

2. General

3. Identificación

4. Feed Google Merchant

5. Manifest UCP

6. Multi-tienda

7. Debug

Pestaña UCP en la ficha de producto

Tareas que NO hace el módulo (necesitan acción tuya)

Resolución de problemas

El feed devuelve "403 Forbidden"

El manifest devuelve 404 en /.well-known/ucp

Los cambios en producto no se reflejan en el feed

No veo el JSON-LD en mi ficha de producto

Soporte

A · ¿Qué es UCP? (texto comercial — para clientes y ficha del módulo)

Titular

Qué es, en una frase

Por qué importa AHORA

Qué hace este módulo ecom_ucpgoogle

Para qué cliente está pensado

Beneficio comercial concreto

B · Realidad técnica de UCP (resumen)

GitHub oficial — ficheros que vamos a usar como referencia

C · Decisiones cerradas (confirmadas con el usuario)

D · Estructura de ficheros

E · Hooks PrestaShop usados

F · SQL — tablas

G · Panel de configuración admin

General

Identificación

Feed Google Merchant

Manifest UCP

Multi-tienda

Debug

 Wizard de Onboarding UCP (pestaña destacada)

Tab en ficha de producto (PS8 + PS9)

H · Endpoints expuestos

I · Fases de implementación (revisar y aprobar)

Fase 0 — Validación previa  (esta fase)

Fase 1 — Esqueleto + instalación + tablas + panel vacío

Fase 2 — UCPDataCollector + Feed XML Google Shopping

Fase 3 — JSON-LD en ficha producto

Fase 4 — Manifest /.well-known/ucp

Fase 5 — Capacidad Catalog REST

Fase 6 — Skill de registro + docs + i18n

I-bis · Roadmap Opción C — Checkout completo UCP

Requisitos externos previos a Opción C (verificados por el wizard, sección G)

Fase C1 — Identity Linking (OAuth 2.0 server)

Fase C2 — Cart capability

Fase C3 — Checkout sessions

Fase C4 — Payment + Order placement

Fase C5 — Order capability + webhooks salientes

Fase C6 — Firmas RFC 9421 + JWK rotation

Fase C7 — Payment Token Exchange capability

Fase C8 — Conformance + enrollment Google + sandbox final

Tablas SQL adicionales (Opción C)

J · Riesgos vivos

K · Confirmación final pendiente para arrancar Fase 1

L · Fuentes

Qué hace

Instalación

Endpoints expuestos

Documentación

Recursos UCP oficiales

1. Arquitectura

2. Tablas

ucp_product

ucp_keys

ucp_log

3. Flujos

3.1 Manifest /.well-known/ucp

3.2 Catalog GET /ucp/catalog/{id}

3.3 Feed XML

3.4 JSON-LD ficha producto

4. Reglas críticas aplicadas

5. Hooks registrados

6. Versionado UCP

7. Roadmap C (Checkout)

Extender el UCPDataCollector

Añadir una capacidad UCP

addNewFeatures()

Tests manuales

Validar feed

Validar manifest

Validar Catalog

Conformance tests UCP

Convenciones de código

¿Qué hace este módulo?

Convierte tu PrestaShop en una tienda visible para los **agentes de inteligencia artificial** (Gemini de Google, AI Mode, Perplexity, ChatGPT...), permitiéndoles consultar tu catálogo en tiempo real y, cuando Google abra el botón "Buy" en Europa, comprar directamente desde la conversación.

Instalación rápida

1. **Subir** la carpeta `ecom_ucpgoogle/` a `/modules/` de tu PrestaShop.
2. **Back-Office** → **Módulos**: buscar "Ecom UCP Google" y pulsar **Instalar**.
3. **Configurar** entrando en "Configurar" del módulo.

Pestañas del panel

1. Onboarding

Aquí verás el wizard de 11 pasos guiados. Cada paso te indica qué falta y dónde hacerlo. *(En esta versión se muestra el placeholder; el wizard completo se incorpora en una actualización siguiente.)*

2. General

Activación del módulo y opciones globales.

3. Identificación

- **País de origen por defecto**: `ES` (España) en clientes españoles. Configurable.
- **Condición**: `Nuevo` por defecto. PrestaShop ya tiene este campo y el módulo lo respeta.
- **GTIN preferente**: `Auto` recomendado. En España casi siempre se usa EAN13. UPC se usa solo en productos americanos.

4. Feed Google Merchant

- **Merchant ID**: el identificador de tu cuenta Google Merchant Center.
- **native_commerce por defecto**: márcalo si quieres que TODOS tus productos estén disponibles para el botón "Buy" de Gemini cuando Google abra el mercado.
- **URL del feed**: copia la URL y pégala en Google Merchant Center → Productos → Subir un feed → Recogida programada.
- **Rotar token**: cambia el token de seguridad. La URL antigua dejará de funcionar inmediatamente.

5. Manifest UCP

- **Versión UCP**: por defecto `2026-04-08`. Cuando Google publique versiones nuevas, podrás actualizar aquí sin reinstalar.
- **Modo de routing**:

- **htaccess (recomendado):** copia el snippet que el módulo te da y pégalo al final de tu fichero `.htaccess` en la raíz de la tienda. Es lo más fiable.
- **dispatcher (fallback):** si no puedes editar el `.htaccess`, el módulo intercepta la petición vía PrestaShop. Más sencillo pero depende del routing interno.
- **URL pública del manifest:** `https://tudominio.com/.well-known/ucp`. Esa es la dirección "oficial" UCP.
- **URL fallback:** siempre disponible aunque el `.htaccess` esté mal configurado. Útil para tests.
- **Rotar clave de firma:** genera una clave nueva. Las antiguas dejan de ser válidas.

6. Multi-tienda

Si tu PrestaShop tiene varias tiendas, cada una tiene su manifest UCP independiente. Selecciona la tienda activa en la cabecera del Back-Office y configura el módulo desde allí.

7. Debug

- **Activar logs:** enciende el modo debug. Crea ficheros `.log` en `/modules/ecom_ucpgoogle/logs/` con todos los eventos UCP.
- **Ver últimos logs:** muestra los últimos 50 eventos en pantalla.
- **Vaciar log:** borra todos los logs en BD (los ficheros físicos no se tocan).

Pestaña UCP en la ficha de producto

En cada producto del Back-Office encontrarás una pestaña **UCP Google** con:

- **Native Commerce:** heredar / sí / no.
- **Condición:** heredar de PrestaShop / nuevo / reacondicionado / usado.
- **País de origen:** dejar vacío para usar el del módulo (ES) o poner otro código ISO-2 (`FR`, `DE`, `IT` ...).
- **GTIN override:** solo si el `EAN13` del producto no es correcto, pon aquí el GTIN real.
- **Consumer notice:** aviso para productos regulados (alimentación, cosmética, electrónica...).

Tareas que NO hace el módulo (necesitan acción tuya)

1. **Crear cuenta Google Merchant Center** y verificar tu dominio. ([Web oficial](#))
2. **Subir el feed** a Google Merchant Center con la URL que te da el módulo.
3. **Pedir acceso UCP a Google** vía el formulario oficial cuando esté disponible para tu país.
4. **Crear cuenta Google Pay & Wallet Console** (solo si activas el addon Checkout en el futuro).

Resolución de problemas

El feed devuelve "403 Forbidden"

Token incorrecto. Vuelve al panel → Feed → copia la URL completa.

El manifest devuelve 404 en `/.well-known/ucp`

Modo `htaccess` no aplicado. Cambia a modo `dispatcher` en el panel → Manifest, o copia bien el snippet `.htaccess`.

Los cambios en producto no se reflejan en el feed

La caché del feed es de 1 hora. Para forzar refresco, abre la URL del feed añadiendo `&refresh=1`.

No veo el JSON-LD en mi ficha de producto

Revisa el código fuente HTML de la página (Ctrl+U) buscando `ld+json`. Si no aparece, comprueba que el módulo está activo y que el hook `displayHeader` está registrado en el módulo.

Soporte

 ecomseo@gmail.com — Ecom Experts

ecom_ucpgoogle — Módulo PrestaShop UCP (Universal Commerce Protocol)

Estado: investigación + decisiones cerradas. Pendiente confirmación final para empezar Fase 1. **Fecha:** 2026-05-08 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 **GitHub UCP oficial:** [Universal-Commerce-Protocol/ucp](#) · [samples](#) **Spec actual:** 2026-04-08 (formato YYYY-MM-DD) · [ucp.dev](#)

A · ¿Qué es UCP? (texto comercial — para clientes y ficha del módulo)

Titular

UCP — Universal Commerce Protocol: el nuevo estándar de Google para vender directamente desde la IA. Tu tienda PrestaShop, visible y comprable dentro de **Gemini, Google AI Mode** y otros agentes de IA.

Qué es, en una frase

Un protocolo abierto, anunciado por Google en enero de 2026 junto a Shopify, Etsy, Wayfair, Target, Walmart, Stripe, Visa, Mastercard y más de 20 socios, que permite a los **agentes de inteligencia artificial** comprar productos en tu tienda **sin abandonar la conversación**.

Por qué importa AHORA

- **Cambio de paradigma:** El usuario ya no busca en Google y entra en tu web. Le pregunta a Gemini "necesito unas zapatillas trail talla 42 por menos de 80€" y la IA le **muestra y le compra**.
- **Sin UCP, no apareces.** Las tiendas que no expongan UCP quedarán invisibles en el comercio agéntico.
- **No es opcional a medio plazo.** Es lo que para SEO fueron los datos estructurados Schema.org en 2014.

Qué hace este módulo `ecom_ucpgoogle`

1. **Hace tu PrestaShop "UCP-ready":** publica el manifest oficial `/ .well-known/ucp` que los agentes IA descubren automáticamente.
2. **Expone tu catálogo en tiempo real** a los agentes (precio, stock, atributos, GTIN), no un feed estático del día anterior.
3. **Genera el feed Google Merchant Center** con `native_commerce: true`, requisito para aparecer en el botón "Buy" de Gemini / AI Mode.
4. **Inyecta JSON-LD Schema.org** en cada ficha de producto (compatible cruzado con OpenAI ACP, Bing, Perplexity).
5. **Marca el país de origen** (España por defecto, configurable).
6. **Lee la condición del producto** (nuevo / reacondicionado / usado) directamente de PrestaShop.
7. **Multi-tienda:** un manifest UCP por cada tienda PrestaShop, con su dominio y catálogo independientes.
8. **Seguro:** firmas HMAC en MVP, claves JWK + RFC 9421 en versión avanzada. Token rotatable para el feed.

Para qué cliente está pensado

- E-commerce PrestaShop 8 o 9 que quiera **adelantarse** al comercio agéntico.
- Tiendas que ya invierten en SEO/Google Shopping y quieren proteger esa inversión cuando la búsqueda se mueve a IA.
- Agencias que quieren ofrecer "preparación UCP" como servicio premium a sus clientes.

Beneficio comercial concreto

"Mientras tu competencia sigue optimizando keywords, tu tienda ya estará dentro de las respuestas de Gemini. Cuando la IA recomiende un producto de tu catálogo, lo recomendará con tu precio y tu stock en tiempo real, y el cliente podrá comprarlo sin salir de la conversación."

B · Realidad técnica de UCP (resumen)

UCP **no es** un feed XML Google Shopping con JSON-LD (eso es solo una pieza). Es un **protocolo bidireccional** con:

- **Manifest de descubrimiento:** `GET /.well-known/ucp` → JSON con versión, capacidades, endpoints y claves públicas JWK.
- **4 capacidades core:** Cart · Checkout · Order · Identity Linking (OAuth 2.0). Más **Catalog** añadida posteriormente.
- **3 transportes:** REST · MCP (JSON-RPC) · A2A. Más Embedded.
- **Seguridad:** HTTPS obligatorio, firmas HTTP RFC 9421 con JWK, tokens de pago opacos.
- **Datos:** importes en céntimos (minor units), fechas RFC 3339, naming reverse-domain.
- **Versionado:** YYYY-MM-DD. Actual `2026-04-08`.
- **Botón "Buy" en AI Mode/Gemini:** requiere atributo `native_commerce: true` en el feed Merchant Center y, hoy, cuenta US.

GitHub oficial — ficheros que vamos a usar como referencia

- `Universal-Commerce-Protocol/ucp` — spec, schemas JSON, docs.
- `Universal-Commerce-Protocol/samples` — implementaciones de referencia:
- `rest/python/server/` (FastAPI) — capabilities, checkout sessions, order lifecycle.
- `rest/nodejs/` (Hono + Zod) — shopping, checkout, order.
- `a2a/business_agent/` — TypeScript, agente A2A.

Conformance tests del repo oficial → los ejecutaremos contra nuestro manifest en Fase 4 antes de marcar nada como terminado.

C · Decisiones cerradas (confirmadas con el usuario)

Decisión	Valor
Versiones PrestaShop	8.x y 9.x (compatibilidad dual)
PHP mínimo	7.4 (compatible 8.0/8.1/8.2)
Multi-tienda	Sí , lee config PrestaShop. Un manifest distinto por tienda
Routing <code>/.well-known/ucp</code>	Configurable en admin: <code>htaccess</code> (recomendado) / <code>dispatcher hook</code> (fallback)
<code>\$this->tab</code> (categoría módulo)	<code>advertising_marketing</code> (válido PS8 y PS9)
GTIN	EAN13 por defecto. UPC sólo si está presente y es válido. Configurable
Condition	Se lee del campo <code>condition</code> del producto PrestaShop. Si el producto no lo define, fallback a config global del módulo. Pestaña admin product para editarlo cómodo
País de origen	España por defecto. Configurable globalmente y por producto
PSP	Amplio — agnóstico. El módulo no integra pagos en MVP, eso es Opción C / fase futura
Alcance MVP	Opción B (Manifest + Catalog + Feed + JSON-LD). Cart/Checkout/Order quedan para fase futura
Modo debug log	Apagado por defecto, activable en config (regla CLAUDE.md: crea <code>/logs</code> y <code>UCPLogger</code>)

D · Estructura de ficheros

```

ecom_ucpgoogle/
├── ecom_ucpgoogle.php           # Clase Ecom_Ucpgoogle (CamelSnakeCase regla CLAUDE.md)
├── config.xml
├── index.php
├── logo.png
├── README.md
├── classes/
│   ├── index.php
│   ├── UCPCDataCollector.php    # Datos producto (precio, stock, gtin, condition, country)
│   ├── UCPCManifestBuilder.php  # /.well-known/ucp por tienda
│   ├── UCPCCatalogResponder.php # Capacidad Catalog REST
│   ├── UCPCFeedXmlBuilder.php   # Feed Google Shopping con native_commerce
│   ├── UCPCSigner.php           # HMAC (MVP) / JWK + RFC 9421 (futuro)
│   ├── UCPCRouter.php           # Decide htaccess vs dispatcher
│   └── UCPCLogger.php           # Logs opcionales
├── controllers/
│   ├── index.php
│   ├── front/
│   │   ├── index.php
│   │   ├── manifest.php        # Sirve /.well-known/ucp (vía rewrite o hook)
│   │   ├── catalog.php         # GET /ucp/catalog/{id}
│   │   └── feed.php            # XML Google Shopping
│   └── admin/
│       └── AdminUcpGoogleConfigController.php # (si necesitamos tab dedicado)
├── views/
│   ├── index.php
│   ├── templates/
│   │   ├── admin/
│   │   │   ├── configure.tpl    # Panel general
│   │   │   └── product_tab.tpl  # Tab "UCP" en ficha producto admin
│   │   └── hook/
│   │       └── header_jsonld.tpl # JSON-LD Product + Offer
│   ├── css/admin.css
│   └── js/admin.js
├── sql/
│   ├── install.sql
│   └── uninstall.sql
├── translations/
├── htaccess_snippet.txt         # Fragmento listo para copiar al .htaccess raíz
├── logs/                        # Sólo si debug ON
└── docs/
    ├── tecnico.md              # 3 docs obligatorios regla CLAUDE.md
    ├── desarrollo.md
    └── usuario.md

```

Nota multi-tienda: El manifest se sirve en el dominio canónico de cada shop. Si dos tiendas comparten dominio pero distinto path, el manifest se sirve en el path de cada tienda (/tienda1/.well-known/ucp no es estándar UCP — habría que usar subdominios). Documentar limitación.

E · Hooks PrestaShop usados

Hook	Para qué
<code>displayHeader</code>	Inyectar JSON-LD Product en <code>ProductController</code>
<code>actionDispatcherBefore</code>	Interceptar <code>/.well-known/ucp</code> (modo dispatcher)
<code>actionProductUpdate</code> / <code>actionProductAdd</code> / <code>actionProductDelete</code>	Invalidar caché feed/catalog
<code>actionObjectStockAvailableUpdateAfter</code>	Refrescar stock UCP
<code>displayAdminProductsExtra</code> (PS8) / <code>displayAdminProductsMainStepLeftColumnBottom</code> (PS9)	Tab UCP en ficha admin producto
<code>actionAdminControllerSetMedia</code>	Assets panel
<code>actionAdminProductsControllerSaveBefore</code> (PS9 simfony)	Guardar campos UCP del tab producto

Todos los hooks van con `public function hookXxx(array $params) { } .`
`isUsingNewTranslationSystem() return true;` añadido (regla CLAUDE.md).

F · SQL — tablas

```
-- Configuración por producto (override de globales)
CREATE TABLE IF NOT EXISTS `PREFIX_ucp_product` (
  `id_product` INT UNSIGNED NOT NULL,
  `id_shop` INT UNSIGNED NOT NULL,
  `native_commerce` TINYINT(1) NOT NULL DEFAULT 1,
  `condition` VARCHAR(20) DEFAULT NULL,          -- new / refurbished / used
  `country_origin` VARCHAR(2) DEFAULT NULL,      -- ISO 3166-1 alpha-2
  `gtin_override` VARCHAR(20) DEFAULT NULL,
  `consumer_notice` TEXT DEFAULT NULL,
  PRIMARY KEY (`id_product`, `id_shop`)
) ENGINE=ENGINE_TYPE DEFAULT CHARSET=utf8mb4;

-- Claves de firma (HMAC MVP, JWK futuro)
CREATE TABLE IF NOT EXISTS `PREFIX_ucp_keys` (
  `id_key` INT UNSIGNED AUTO_INCREMENT,
  `id_shop` INT UNSIGNED NOT NULL,
  `kid` VARCHAR(64) NOT NULL,
  `algo` VARCHAR(20) NOT NULL,                   -- HS256, ES256, RS256
  `secret` TEXT NOT NULL,                        -- cifrado
  `public_jwk` TEXT DEFAULT NULL,
  `active` TINYINT(1) NOT NULL DEFAULT 1,
  `created_at` DATETIME NOT NULL,
  PRIMARY KEY (`id_key`)
) ENGINE=ENGINE_TYPE DEFAULT CHARSET=utf8mb4;

-- Log opcional
CREATE TABLE IF NOT EXISTS `PREFIX_ucp_log` (
  `id_log` INT UNSIGNED AUTO_INCREMENT,
  `id_shop` INT UNSIGNED NOT NULL,
  `level` VARCHAR(10) NOT NULL,
  `channel` VARCHAR(40) NOT NULL,                 -- manifest, catalog, feed, signer
  `message` TEXT NOT NULL,
  `context` LONGTEXT DEFAULT NULL,               -- JSON
  `created_at` DATETIME NOT NULL,
  PRIMARY KEY (`id_log`),
  KEY `idx_shop_chan` (`id_shop`, `channel`)
) ENGINE=ENGINE_TYPE DEFAULT CHARSET=utf8mb4;
```

Mantenimiento de schema: **NO** ficheros upgrade/. Función `addNewFeatures()` ejecutada al entrar en config (regla CLAUDE.md).

G · Panel de configuración admin

General

- Activo (bool).

- Modo de routing manifest: **htaccess** (muestra snippet a copiar) / **dispatcher** (automático).
- Botón "Probar manifest" → cURL local que valida `/.well-known/ucp` responde 200 con JSON correcto.

Identificación

- **País de origen por defecto:** select países (default `ES`).
- **GTIN preferente:** EAN13 / UPC / Auto (recomendado).
- **Condición por defecto:** `new` / `refurbished` / `used`.

Feed Google Merchant

- Token feed (rotable).
- URL feed (display + copiar).
- Activar `native_commerce: true` para: todos los productos / categorías / selección manual.
- Campo `consumer_notice` global (opcional, para productos regulados).
- TTL caché feed.

Manifest UCP

- Versión UCP soportada (default `2026-04-08`).
- URL manifest (display + copiar).
- Capacidades activas: ☒ Catalog · ☐ Cart (futuro) · ☐ Checkout (futuro) · ☐ Order (futuro) · ☐ Identity Linking (futuro).
- Algoritmo firma: HMAC-SHA256 (MVP) / ES256 / RS256 (avanzado).
- Botón "Generar/rotar clave".

Multi-tienda

- Tabla con cada Shop y su URL manifest individual.
- Override de configuración por tienda.

Debug

- Modo debug log ON/OFF.
- Botón "Ver últimos 50 logs".
- Botón "Vaciar log".

Wizard de Onboarding UCP (pestaña destacada)

Para qué: ningún cliente puede activar UCP sin completar trámites externos con Google. Esta pestaña es **una checklist guiada paso a paso** que verifica cada requisito automáticamente cuando es posible y abre el enlace exacto cuando hay que hacerlo a mano.

Estructura de la pestaña — cada paso muestra: estado ( /  / ) , explicación corta, botón "Comprobar ahora", botón "Abrir enlace oficial", y nota informativa.

#	Paso	Cómo lo verifica el módulo	Acción para el cliente
1	HTTPS válido en el dominio	cURL al propio dominio + chequeo certificado	Enlace a SSL Labs si falla
2	Manifest accesible en <code>/.well-known/ucp</code>	cURL a la URL real, valida JSON y firma	Botón "Generar snippet htaccess" o "Activar modo dispatcher"
3	Cuenta Google Merchant Center activa	Pide al usuario el Merchant ID y llama API GMC para validar	Enlace merchants.google.com
4	Feed Google Shopping aprobado	Consulta estado del feed vía GMC API si hay credenciales	Botón "Subir feed ahora"
5	Atributo <code>native_commerce:true</code> activo en al menos 1 producto	Lectura de <code>ucp_product</code>	Enlace al panel de configuración masiva
6	País de tienda configurado	Lee <code>Configuration::get('PS_SHOP_COUNTRY_ID')</code>	Aviso: España OK / EU pendiente / US listo
7	Interest form UCP enviado a Google	Manual — checkbox "Lo he enviado el [fecha]"	Enlace al formulario oficial Google UCP
8	Cuenta Google Pay & Wallet Console	Manual — pide el Merchant ID de Google Pay	Enlace pay.google.com/business/console
9	PSP compatible con tokens Google Pay	Detecta el módulo de pago activo y compara contra lista interna (Stripe, Adyen, Redsys-MIT, PayPal, Worldpay...)	Aviso si el PSP no está en la lista, con sugerencias
10	Sandbox UCP probado (sólo si se va a Opción C)	Test automático: crea cart → checkout → order ficticia	Muestra log resultado
11	Aprobación final de Google	Manual — el cliente marca cuando Google le confirma	Avisa que el módulo entrará en modo producción

Cabecera del wizard: muestra un porcentaje "UCP Readiness: 60%" calculado sobre los 11 pasos. Cada paso tiene un texto explicativo de 2-3 líneas en castellano sencillo (no técnico).

Modos de operación según wizard: - **0-30%:** Modo **"Preparación"** — solo se exponen feed XML y JSON-LD. El manifest devuelve 503. - **31-70%:** Modo **"Discovery"** — manifest activo + Catalog. El cliente ya es descubrible por agentes IA en lectura. - **71-99%:** Modo **"Pre-producción"** — Cart + Checkout activos en sandbox. - **100%:** Modo **"Producción"** — Checkout real con Google Pay.

Botón "Exportar informe PDF" — genera un PDF con el estado de los 11 pasos para que el cliente lo envíe a su gestor o a su agencia.

Tab en ficha de producto (PS8 + PS9)

- Override de `native_commerce` (sí/no/heredar).
- Override de `condition` (new/refurbished/used/heredar).
- Override de país de origen.
- Override GTIN.
- `consumer_notice` específico del producto.

H · Endpoints expuestos

Endpoint	Método	Para qué	Auth
<code>/.well-known/ucp</code>	GET	Manifest descubrimiento	Pública, firmada con JWK del shop
<code>/module/ecom_ucpgoogle/feed?token=...</code>	GET	XML Google Shopping	Token HMAC corto en query
<code>/module/ecom_ucpgoogle/catalog?id=...</code>	GET	Producto en tiempo real (UCP Catalog)	Firma HTTP en respuesta

I · Fases de implementación (revisar y aprobar)

Fase 0 — Validación previa (esta fase)

Decisiones cerradas. Pendiente: ¿**confirmas el GitHub** `Universal-Commerce-Protocol/ucp` **o tienes otro?** y ¿**arrancamos Fase 1?**

Fase 1 — Esqueleto + instalación + tablas + panel vacío

- `ecom_ucpgoogle.php` — install/uninstall, hooks declarados, `addNewFeatures()`, `isUsingNewTranslationSystem`.
- `config.xml`, `index.php` recursivos, `logo.png`.
- SQL install/uninstall con tablas `ucp_product`, `ucp_keys`, `ucp_log`.
- Panel admin vacío (estructura tabs).
- Tab UCP en ficha de producto admin (PS8 vía `displayAdminProductsExtra`, PS9 vía hook Sf).
- `UCPLogger` funcional pero apagado por defecto.
- **Criterio de aceptación:** instala y desinstala limpio en PS8 y PS9, sin warnings PHP 7.4 ni 8.x.

Fase 2 — UCPDataCollector + Feed XML Google Shopping

- `UCPDataCollector::getProductData($id_product, $id_lang, $id_shop)` con todo el mapeo (precio impuestos incluidos, stock, gtin EAN13/UPC, condition, country, brand, image large_default).
- `UCPFeedXmlBuilder` genera XML compatible Google Shopping con namespace `g:`.
- `controllers/front/feed.php` con token HMAC.
- Caché archivo con TTL.
- `native_commerce: true` aplicado según config.
- **Criterio:** validar feed con [Google Merchant Center Feed Tester](#).

Fase 3 — JSON-LD en ficha producto

- Hook `displayHeader` solo en `ProductController`.
- Schema.org `Product` con `Offer`, `gtin13` / `gtin12`, `availability`, `priceCurrency`, `countryOfOrigin`, `condition`, `brand`, `image`.
- **Criterio:** validar con [Google Rich Results Test](#) y [Schema.org Validator](#).

Fase 4 — Manifest `/well-known/ucp`

- `UCPManifestBuilder` JSON con: `version: 2026-04-08`, transportes, capacidades declaradas (Catalog inicial), `signing_keys` JWK públicas.
- `UCPRouter` con dos modos: htaccess (genera snippet) / dispatcher (hook `actionDispatcherBefore`).
- HTTPS obligatorio, `Cache-Control: max-age=60`.
- Multi-tienda: un manifest por dominio/shop.
- **Criterio:** ejecutar conformance tests del repo [Universal-Commerce-Protocol/ucp](#).

Fase 5 — Capacidad Catalog REST

- `controllers/front/catalog.php`.
- `UCPCatalogResponder` devuelve producto en tiempo real: precio (céntimos), stock, atributos, GTIN, country, condition.
- `UCPSigner` firma respuestas (HMAC en MVP).
- Headers UCP correctos.
- **Criterio:** validación contra samples Node.js de [Universal-Commerce-Protocol/samples](#).

Fase 6 — Skill de registro + docs + i18n

- 3 docs `.md` (técnico, desarrollo, usuario).
- Aplicar skill `prestashop_connect_gmartos` para registrar el módulo.
- Traducciones ES/EN.
- README de venta.
- **Criterio:** docs revisadas, módulo listado en herramienta externa.

I-bis · Roadmap Opción C — Checkout completo UCP

Premisa de arquitectura: aunque las Fases 1-6 del MVP entregan Opción B, el módulo se **diseña desde el primer commit** pensando en C. Las clases base (UCPSigner, UCPRouter, UCPLoader, manifest builder) se crean abstrayendo lo que C necesitará — sin sobre-ingeniería pero sin atajos que haya que reescribir luego. **El módulo entero es un sistema UCP, no un feed con extras.**

Requisitos externos previos a Opción C (verificados por el wizard, sección G)

1. Cuenta Google Pay & Wallet Console activa.
2. Merchant Center sin violaciones de política.
3. Interest form UCP enviado y aceptado por Google.
4. PSP compatible con tokens Google Pay.
5. HTTPS sólido + dominio canónico fijo.
6. Cumplimiento PCI-DSS SAQ A mínimo.
7. Restricción geográfica: hoy sólo EE.UU. — clientes ES quedan en modo "preparado" hasta apertura EU.

Fase C1 — Identity Linking (OAuth 2.0 server)

- Endpoints: `/oauth/authorize`, `/oauth/token`, `/oauth/refresh`, `/oauth/revoke`.
- Pantalla de consentimiento con login PrestaShop.
- Vinculación `OAuthClient` ↔ `ucp_oauth_tokens` ↔ `Customer`.
- Scopes: `cart:rw`, `checkout:rw`, `order:read`.
- Refresh tokens con rotación, expiración configurable.
- **Aceptación:** flujo OAuth 2.0 completo válido contra RFC 6749 + suite Postman.

Fase C2 — Cart capability

- `POST /ucp/cart`, `GET /ucp/cart/{id}`, `POST /ucp/cart/{id}/items`, `PATCH ...`, `DELETE ...`.
- Sync 1:1 con `Cart` PrestaShop, sin segundo carrito paralelo.
- Stock check tiempo real, lock blando durante checkout.
- Multi-currency, importes en céntimos, fechas RFC 3339.
- **Aceptación:** carrito UCP visible también en `/cart` web del cliente si abre la tienda.

Fase C3 — Checkout sessions

- `POST /ucp/checkout/session` desde un `cart_id`.
- `PUT /shipping-address` → recálculo carriers + tax con motor PrestaShop.
- `PUT /shipping-method`, `POST /apply-discount`.
- Lock de stock con TTL (default 15 min, configurable).
- **Reutiliza** lógica de impuestos/transportistas/promos PS sin duplicar.
- **Aceptación:** totales idénticos a `OrderController` PS para mismo carrito.

Fase C4 — Payment + Order placement

- Recepción token Google Pay opaco vía Payment Token Exchange.
- **Adapter pattern por PSP:**
- `UCPPaymentAdapter_Stripe` (clase base).
- `UCPPaymentAdapter_Adyen`.
- `UCPPaymentAdapter_Redsys` (con MIT).
- `UCPPaymentAdapter_PayPal`.
- Otros PSP → adapter custom documentado.
- Creación de pedido vía `PaymentModule::validateOrder()` PS — mantiene compatibilidad con todos los hooks/módulos del cliente (facturación, almacén, contabilidad).
- Devuelve `order_id` al agente.
- **Aceptación:** pedido aparece en BO PS exactamente como un pedido normal, todos los hooks ejecutados.

Fase C5 — Order capability + webhooks salientes

- `GET /ucp/order/{id}`.
- Webhooks a Google al cambiar estado: `payment_confirmed`, `shipped` (con tracking), `delivered`, `cancelled`, `refunded`.
- Hooks PS: `actionOrderStatusUpdate`, `actionObjectOrderHistoryAddAfter`.
- Cola de reintentos con backoff exponencial en tabla `ucp_webhook_queue`, dead-letter tras N fallos.
- Firma HMAC saliente (sustituida en C6 por RFC 9421).
- **Aceptación:** estado en Google espejo de PS con latencia < 60 s.

Fase C6 — Firmas RFC 9421 + JWK rotation

- Reemplazo de HMAC por HTTP Message Signatures completo.
- Algoritmos `ES256` / `RS256`.
- Rotación JWK con `kid`, ventana de gracia 30 días.
- Verificación de firmas entrantes de Google.
- **Aceptación:** conformance tests del repo [Universal-Commerce-Protocol/ucp](#) en verde.

Fase C7 — Payment Token Exchange capability

- Capacidad UCP dedicada al intercambio de tokens de pago.
- Coordinación entre token Google Pay y el PSP del cliente.
- Manejo de **3-D Secure / SCA (PSD2)** cuando aplica EU.
- **Aceptación:** transacción real en sandbox del PSP elegido.

Fase C8 — Conformance + enrollment Google + sandbox final

- Suite completa de conformance tests UCP (Python, ejecutar manualmente, documentado).
- Acompañar al cliente en alta Google Pay & Wallet Console.

- Sandbox testing con Google (orders ficticias punta a punta).
- Documentación operativa: reconciliación si Google paga y PS falla, devoluciones, cancelaciones.
- **Aceptación:** Google certifica al merchant y le habilita botón "Buy" real.

Tablas SQL adicionales (Opción C)

- `ucp_oauth_clients` — clientes OAuth registrados.
- `ucp_oauth_tokens` — access/refresh tokens.
- `ucp_cart_session` — locks de stock.
- `ucp_webhook_queue` — webhooks pendientes con reintentos.
- `ucp_order_link` — mapeo `id_order` PS ↔ `ucp_order_id`.

J · Riesgos vivos

1. **Spec UCP en evolución.** La versión `2026-04-08` puede mover. Estrategia: anclar versión en `Configuration::get('UCP_VERSION')` editable, y avisar en admin si hay nueva.
2. **`/well-known/` y multi-tienda en mismo dominio:** UCP espera el manifest en la raíz del host. Si el cliente tiene PS multi-tienda con paths (`/es/`, `/fr/`), una de las dos tiendas no podrá publicar manifest oficial. Documentar y recomendar subdominios.
3. **PS9 + Symfony:** `displayAdminProductsExtra` ya no existe igual; usar el hook Sf nuevo. Hay que probar en PS9 real.
4. **Google Merchant Center US-only en MVP:** el botón Buy de Gemini funciona solo en cuentas US a fecha de 2026-05. El módulo prepara los datos correctamente para cuando se abra a EU/ES.
5. **Conformance tests del repo UCP están en Python:** los ejecutaremos manualmente desde una shell, no integrados. Documentar en `docs/desarrollo.md` cómo correrlos.

K · Confirmación final pendiente para arrancar Fase 1

- ☒ Alcance: Opción B (Manifest + Catalog + Feed + JSON-LD).
- ☒ PrestaShop 8 y 9.
- ☒ Multi-tienda con manifest por shop.
- ☒ Routing configurable htaccess/dispatcher.
- ☒ EAN13/UPC configurable, fallback EAN13.
- ☒ Condition desde producto + tab admin + fallback global.
- ☒ País origen España default + por producto.
- ☒ PSP agnóstico.
- ☒ `$this->tab = 'advertising_marketing'`.
- ☒ **GitHub de referencia:** ¿es el oficial [Universal-Commerce-Protocol/ucp](#) que has detectado o tienes otro repo en mente?

- [? ¿Arrancamos Fase 1?](#)
-

L · Fuentes

- [ucp.dev — spec 2026-04-08](#)
- [GitHub Universal-Commerce-Protocol/ucp](#)
- [GitHub Universal-Commerce-Protocol/samples](#)
- [Google for Developers — UCP](#)
- [Google Merchant Center Help — UCP checkout](#)
- [Under the Hood: UCP — Google Developers Blog](#)
- [Google Cloud — agentic commerce](#)
- [Shopify Engineering — Building UCP](#)
- [PrestaShop devdocs 9 — admin tabs](#)

ecom_ucpgoogle

Universal Commerce Protocol (UCP) connector for PrestaShop 8 and 9.

Convierte tu PrestaShop en una tienda compatible con UCP — el estándar abierto de Google para comercio agéntico (Gemini, AI Mode y demás agentes IA).

- Versión: 1.0.0
- Spec UCP soportada: 2026-04-08
- Compatible: PrestaShop 8.x · PrestaShop 9.x
- PHP: 7.4 — 8.x
- Licencia: AFL-3.0
- Autor: Ecom Experts (ecomyseu@gmail.com)

Qué hace

1. Publica `/.well-known/ucp` (manifest UCP firmado).
2. Expone capacidad **Catalog** REST — datos de producto en tiempo real.
3. Genera el feed **Google Shopping** con atributo `native_commerce`.
4. Inyecta **JSON-LD Schema.org** en cada ficha de producto.
5. Pestaña **UCP** en la ficha de producto admin (override de `native_commerce`, `condition`, `GTIN`, `país`, `consumer_notice`).
6. Multi-tienda nativa.

Instalación

1. Sube la carpeta `ecom_ucpgoogle/` a `/modules/` de tu PrestaShop.
2. Back-Office → Módulos → busca "Ecom UCP Google" → Instalar.
3. Configurar: - **Identificación**: país por defecto (ES), condición, GTIN preferente. - **Feed**: copia la URL del feed para Google Merchant Center. - **Manifest**: elige modo `htaccess` (recomendado) o `dispatcher`.

Endpoints expuestos

Endpoint	Para qué
<code>https://tudominio.com/.well-known/ucp</code>	Manifest descubrimiento
<code>/index.php?fc=module&module=ecom_ucpgoogle&controller=feed&token=XXX</code>	Feed XML Google Shopping
<code>/index.php?fc=module&module=ecom_ucpgoogle&controller=catalog&id=PROD_ID</code>	Capacidad Catalog (UCP)

Con friendly URLs activas: - `/well-known/ucp` - `/ucp/feed.xml` - `/ucp/catalog/{id}`

Documentación

- [docs/tecnico.md](#) — Arquitectura técnica.
- [docs/desarrollo.md](#) — Guía de desarrollo y extensión.
- [docs/usuario.md](#) — Manual del administrador.

Recursos UCP oficiales

- [ucp.dev](#)
- [GitHub Universal-Commerce-Protocol](#)
- [Google Developers UCP](#)

ecom_ucpgoogle · Documentación técnica

1. Arquitectura

ecom_ucpgoogle/	
├ ecom_ucpgoogle.php	Clase principal Ecom_Ucpgoogle (Module)
├ config.xml	
├ classes/	
│ └ UCPLogger.php	Logging condicional BD + fichero
│ └ UCPCollector.php	Extrae datos UCP de un producto PS
│ └ UCPFeedXmlBuilder.php	XML Google Shopping (namespace g:)
│ └ UCPJsonLdBuilder.php	Schema.org Product+Offer JSON-LD
│ └ UCPSigner.php	HMAC-SHA256 + gestión de claves
│ └ UCManifestBuilder.php	JSON manifest UCP
│ └ UCPCatalogResponder.php	Mapeo a UCP catalog (reverse-domain)
│ └ UCPRouter.php	Routing /.well-known/ucp
├ controllers/front/	
│ └ manifest.php	Sirve /.well-known/ucp
│ └ catalog.php	GET /ucp/catalog/{id}
│ └ feed.php	GET feed.xml (token HMAC)
├ views/templates/admin/	
│ └ configure.tpl	Panel admin con tabs
│ └ product_tab.tpl	Tab UCP en ficha producto
│ └ gmartos_update.tpl	Aviso actualización Gmartos
└ sql/	
├ install.sql	ucp_product, ucp_keys, ucp_log
└ uninstall.sql	

2. Tablas

ucp_product

Override por producto y por tienda. Clave primaria compuesta (`id_product`, `id_shop`) . Campos: `native_commerce`, `condition_value`, `country_origin`, `gtin_override`, `consumer_notice`, `date_upd` .

ucp_keys

Claves de firma. `kid` único, `algo` (HS256 MVP, ES256 / RS256 futuro), `secret` (HMAC) y `public_jwk` (JWK opcional).

ucp_log

Log condicional. Solo se escribe si `ECOM_UCP_DEBUG = 1` . Errores se persisten siempre.

3. Flujos

3.1 Manifest `/well-known/ucp`

1. Cliente solicita `https://shop.tld/.well-known/ucp`.
2. Modo **htaccess**: Apache reescribe a `index.php?fc=module&module=ecom_ucpgoogle&controller=manifest`.
3. Modo **dispatcher**: `hookActionDispatcherBefore` detecta la URI y responde directamente.
4. `UCPManifestBuilder::buildJson()` genera el JSON.
5. `UCPSigner::sign()` añade firma HMAC en headers `X-UCP-Signature*`.

3.2 Catalog `GET /ucp/catalog/{id}`

1. `UCPCatalogResponder::getCatalogItem()` llama a `UCPDataCollector::getProductData()`.
2. Mapea al formato UCP: `price.amount_minor` (céntimos), `availability.state` (`in_stock` / `out_of_stock`), `identifiers.gtin13` / `gtin12` / `mpn` / `sku`, `last_modified` RFC 3339.
3. Firma HMAC en headers de respuesta.

3.3 Feed XML

1. Token HMAC validado vía `hash_equals`.
2. Caché en `/logs/cache/feed_shop_{id}_lang_{id}.xml`, TTL 3600s, refresco con `?refresh=1`.
3. Itera todos los productos activos paginados (300 por bucle).
4. Atributo crítico: `<g:native_commerce>true</g:native_commerce>`.

3.4 JSON-LD ficha producto

1. `hookDisplayHeader` solo en `ProductController` (chequea con `get_class()`, no con `::class`).
2. `UCPJsonLdBuilder::renderScript()` devuelve `<script type="application/ld+json">...</script>`.
3. Se devuelve directamente desde el hook (sin Smarty, evita `nofilter`).

4. Reglas críticas aplicadas

- **NO LIMIT 1**: usamos `getRow()` / `getValue()` (PrestaShop ya lo añade) o paginado real.
- **NO Composer**: `checkGmartosUpdate()` usa cURL nativo, no Guzzle.
- **NO `$controller::class`**: `is_object($controller) ? get_class($controller) : ''`.
- **NO `nofilter`**: el JSON-LD se inyecta desde PHP, no desde la plantilla.
- `addNewFeatures()` en lugar de carpeta `upgrade/`.
- **Headers en TODOS los ficheros** (PHP, JS, CSS, TPL, SQL) sin líneas en blanco antes/después de `<?php`.
- `index.php` **recursivo** en cada subdirectorio con redirect 301.
- `isUsingNewTranslationSystem(): true`.
- `pSQL()` y **type-casting** para entrada en SQL manual.
- **Skill `prestashop_connect_gmartos`** integrado (registro semanal en modules.gmartos.es).

5. Hooks registrados

Hook	Estado
<code>displayHeader</code>	✓ JSON-LD producto
<code>actionDispatcherBefore</code>	✓ Modo dispatcher manifest
<code>actionProductUpdate</code> / <code>actionProductAdd</code> / <code>actionProductDelete</code>	✓ Persistencia + cache invalidation
<code>actionObjectStockAvailableUpdateAfter</code>	✓ Cache invalidation
<code>actionAdminControllerSetMedia</code>	✓ Assets panel admin
<code>displayAdminProductsExtra</code>	✓ Tab UCP PS 8
<code>displayAdminProductsMainStepLeftColumnBottom</code>	✓ Tab UCP fallback
<code>actionProductFormBuilderModifier</code>	🕒 PS 9 Symfony (Fase futura)
<code>actionAdminProductsControllerSaveBefore</code>	✓ Save PS 8
<code>moduleRoutes</code>	✓ URLs amistosas

6. Versionado UCP

`ECOM_UCP_VERSION` configurable, default `2026-04-08`. Formato YYYY-MM-DD validado por regex.

7. Roadmap C (Checkout)

Tablas adicionales pendientes: `ucp_oauth_clients`, `ucp_oauth_tokens`, `ucp_cart_session`, `ucp_webhook_queue`, `ucp_order_link`. Ver `info.md` sección I-bis.

ecom_ucpgoogle · Guía de desarrollo

Extender el UCPDataCollector

Si tu cliente tiene atributos extra (composición textil, fecha de fabricación), añade un método propio:

```
public function getProductData($id_product, $id_lang = null, $id_shop = null) {
    $data = parent::getProductData($id_product, $id_lang, $id_shop);
    if ($data === null) return null;
    $data['custom_field'] = 'valor';
    return $data;
}
```

No subclases en módulos hijos: añade el campo en `UCPDataCollector` directamente y el `feed/manifest/jsonld` lo recogerán.

Añadir una capacidad UCP

1. Crear `classes/UCPCapXxxResponder.php` con el responder.
2. Crear `controllers/front/xxx.php`.
3. Declarar en `UCPManifestBuilder::build()` dentro de `capabilities`:

```
'dev.ucp.shopping.xxx' => [
    'version' => '1.0.0',
    'transport' => 'rest',
    'endpoint' => $this->context->link->getModuleLink('ecom_ucpgoogle', 'xxx', [], true),
    'methods' => ['GET'],
],
```

4. Añadir ruta en `hookModuleRoutes()`.

addNewFeatures()

Cuando subas la versión del schema, edita `Ecom_Ucpgoogle::SCHEMA_VERSION` y añade el ALTER en el cuerpo de `addNewFeatures()`. Ejemplo:

```
public function addNewFeatures() {
    $current = (string) Configuration::get('ECOM_UCP_SCHEMA_VERSION');
    if ($current === self::SCHEMA_VERSION) return true;
    try {
        Db::getInstance()->execute('ALTER TABLE `.` . _DB_PREFIX_ . 'ucp_product`
            ADD COLUMN IF NOT EXISTS `new_field` VARCHAR(50) DEFAULT NULL');
        Configuration::updateValue('ECOM_UCP_SCHEMA_VERSION', self::SCHEMA_VERSION);
        return true;
    } catch (\Exception $e) {
        return false;
    }
}
```

NO crear ficheros en `upgrade/` (regla del proyecto).

Tests manuales

Validar feed

- Abrir la URL del feed con `?refresh=1` para forzar regeneración.
- Validador externo: [Schema Markup Validator](#).

Validar manifest

- `curl -i https://shop.tld/.well-known/ucp` debe devolver 200 + cabeceras `X-UCP-Signature*`.
- JSON debe tener `protocol: "ucp"`, `version: "2026-04-08"`, `capabilities`.

Validar Catalog

- `curl https://shop.tld/index.php?fc=module&module=ecom_ucpgoogle&controller=catalog&id=1`.
- Verificar `price.amount_minor` (céntimos) y `availability.state`.

Conformance tests UCP

Repo oficial: <https://github.com/Universal-Commerce-Protocol/ucp>. Los tests están en Python:

```
git clone https://github.com/Universal-Commerce-Protocol/ucp
cd ucp
pip install -r requirements.txt
python -m pytest tests/conformance --target https://shop.tld
```

Convenciones de código

- Prefix de clases UCP: `UCP`.
- Constantes de configuración: `ECOM_UCP_*`.
- Hooks declarados como `public function hookXxx(array $params)`.

- Logger: usar `$this->getLogger()->info|warning|error('canal', 'mensaje', $context)` . Canales habituales: `manifest` , `catalog` , `feed` , `jsonld` , `install` , `product` .